

AMUSEMENT PARK TICKET BOOKING SYSTEM

FINAL PROJECT REPORT

CIS 552 Database Design (Summer 2025)

Advisor: Prof. Ashokkumar Patel

Team: 12

FUTURE ENHANCEMENTS:

The Amusement Park Ticket Booking System is a database-driven application that manages rides, customers, ticket types, and bookings through a structured relational model. The planned use case involves customers selecting rides and ticket types, entering their details, and creating bookings, while administrators manage ride information and view analytics. The MySQL data store paradigm provides a strictly defined schema with enforced relationships, ensuring transactional integrity, accurate capacity tracking, and reliable analytics. CRUD operations are handled via the Node.js/Express backend, with SQL queries performing data validation, joins, and aggregation for reporting.

Future extensions could include multi-ride booking per transaction, auto-refresh analytics, add-ons like Fast-Pass or meal plans, advanced time-slot booking, and online payment integration. While MySQL remains ideal for structured transactional data, these expanded features could be augmented with other paradigms, such as a document store (MongoDB) to manage flexible add-on configurations, or a graph store (Neo4j) to analyse relationships between rides, booking patterns, and visitor preferences for targeted offers and crowd management.

APIs (Backend Endpoints in server.js)

The backend is built using Node.js with Express and exposes the following routes:

Rides:

- GET /rides – Fetch all rides from MySQL.
- POST /rides – Add a new ride (name, description, duration, capacity, ticket_price).
- PUT /rides/:id – Update ride details.
- DELETE /rides/:id – Delete a ride.

Customers:

- POST /customers – Add a new customer (name, email, phone).

Ticket Types:

- GET /ticket-types – Get available ticket types (e.g., Single Ride, Full Day Pass, Group Ticket).

Bookings:

- GET /bookings – Fetch all current bookings.
- POST /bookings – Create a booking (customer_id, ride_id, booking_time, status, ticket_type_id).
- PUT /bookings/:id – Update a booking (e.g., change status).
- DELETE /bookings/:id – Cancel a booking.

Ride Wait Time:

- GET /ride-wait-time/:id – Get estimated wait time for a specific ride.

Analytics:

- GET /analytics – Get total bookings and revenue per ride.

Tools & Technologies Used

Frontend:

- Language: JavaScript (ES6)
- Framework/Library: React (Functional Components + Hooks)
- UI Styling: Basic HTML/CSS (in React components)
- Data Fetching: Native fetch() API to call backend endpoints
- Frontend File: src/App.js

Backend:

- Language: JavaScript (Node.js)
- Framework: Express.js
- Database Library: mysql2 for MySQL connection and queries
- Middleware: cors for Cross-Origin Resource Sharing (frontend ↔ backend communication), body-parser for JSON request body parsing
- Backend File: server.js

Database:

- DBMS: MySQL
- Schema Includes:
 - rides table (ride_id, name, description, duration, capacity, ticket_price)
 - customers table (customer_id, name, email, phone)
 - ticket_types table (ticket_type_id, name, price_modifier)
 - bookings table (booking_id, customer_id, ride_id, booking_time, status, ticket_type_id)
 - Migration File: 20250810164232_shiny_limit.sql (creates tables and seeds ticket_types)

BACKEND CONNECTION TO DATABASE:

Established a connection between the database and the workbench:

```
amusement-park-backend > JS server.js > ...
1  const express = require('express'); //for running the app ; part of the MERN
2  const mysql = require('mysql2');
3  const bodyParser = require('body-parser');
4  const cors = require('cors'); //connection to backend anf front end
5
6  const app = express();
7  app.use(cors());
8  app.use(bodyParser.json());
9
10 // MAKING THE BACKEND CONNECT WITH THE DATABASE
11 const db = mysql.createConnection({
12   host: 'localhost',
13   user: 'root',
14   password: ' ' // Use your password here
15   database: 'amusement_park'
16 });
17 // console log for database connection error
18 db.connect((err) => {
19   if (err) {
20     console.error('DB connection error:', err);
21     process.exit(1);
22   }
23   console.log('Connected to MySQL database amusement_park.');
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS C:\Users\laura\Downloads\project> cd amusement-park-backend
○ PS C:\Users\laura\Downloads\project\amusement-park-backend> npm start

> amusement-park-backend@1.0.0 start
> node server.js

Backend server started on port 3001
Connected to MySQL database amusement_park.
```

Welcome to MySQL Workbench

MySQL Workbench is the official graphical user interface (GUI) tool for MySQL. It allows you to design, create and browse your database schemas, work with database objects and insert data as well as design and run SQL queries to work with stored data. You can also migrate schemas and data from other

[Browse Documentation >](#)

[Discuss on the Forums >](#)

MySQL Connections

Local instance MySQL80	AzureDB
root localhost:3306	shruthiravi3001 db1-summer2025.mysql.database.azure...

Connect to MySQL Server

Please enter password for the following service:

Service: MySQL@localhost:3306

User: root

Password:

☐ Save password in vault

OK Cancel

[Filter conn](#)

Creation of database, tables, User inputs, and EER diagram

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator SQL File 13' customers bookings ticket_types rides Limit to 1000 rows

SCHEMAS

Filter objects

amusement_park

Tables

bookings

customers

rides

ticket_types

Views

Stored Procedures

Functions

company

mydb

sakila

sys

world

Administration Schemas

Information

No object selected

1 USE amusement_park;

2

3 -- Create or update rides table

4 CREATE TABLE IF NOT EXISTS rides (

5 ride_id INT AUTO_INCREMENT PRIMARY KEY,

6 name VARCHAR(100) NOT NULL,

7 description TEXT,

8 duration INT, -- in minutes

9 capacity INT NOT NULL CHECK (capacity >= 0),

10 ticket_price DECIMAL(6,2) NOT NULL DEFAULT 10.00

11);

12

13 -- Create or update customers table

14 CREATE TABLE IF NOT EXISTS customers (

15 customer_id INT AUTO_INCREMENT PRIMARY KEY,

16 name VARCHAR(100) NOT NULL,

17 email VARCHAR(100) NOT NULL,

18 phone VARCHAR(20)

19);

20

21 -- Create ticket_types table

22 CREATE TABLE IF NOT EXISTS ticket_types (

23 ticket_type_id INT AUTO_INCREMENT PRIMARY KEY,

24 name VARCHAR(50) NOT NULL,

25 price_modifier DECIMAL(5,2) NOT NULL COMMENT 'Multiplier for base price, e.g., 1.0 for regular, 0.8 for discount'

26);

27

28 SELECT r.name AS ride_name,

29 COUNT(b.booking_id) AS booking_count,

30 SUM(r.ticket_price * tt.price_modifier) AS total_revenue

31 FROM bookings b

SQL File 13* customers bookings ticket_types rides

Limit to 1000 rows

```
1 • SELECT * FROM amusement_park.customers;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Contents: |

customer_id	name	email	phone
8	sandhya	sandhya@gmail.com	87654321
9	sandy	san@gmail.com	897564321
10	gojo	gojo@gmail.com	123456789
11	SANDHYA	sandhya@gmail.com	9176632388
12	SHRUTHI	shruthi@gmail.com	8765432112
13	SHRUTHI	shruthi@gmail.com	8765432112
14	gojo	gojo@gmail.com	987654321
15	sam	sam@email.com	0864213579
16	sam	sam@email.com	0864213579
17	Gia	Gia@gmail.com	1357864209
18	Gia	Gia@gmail.com	1357864209

SQL File 13* customers bookings ticket_types rides

Limit to 1000 rows

```
1 • SELECT * FROM amusement_park.bookings;
```

2

3

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Contents: |

booking_id	customer_id	ride_id	booking_time	status	ticket_type_id
1	9	1	2025-08-11 05:41:49	Booked	1
4	20	3	2025-08-11 17:25:05	Booked	2
9	25	2	2025-08-12 15:47:42	Booked	3
10	26	3	2025-08-12 15:58:14	Booked	2
11	27	1	2025-08-12 17:53:34	Booked	1

Form Editor

SQL File 13* customers bookings ticket_types rides

1 • SELECT * FROM amusement_park.ticket_types;

Result Grid Filter Rows: Edit Export/Import: Wrap Cell Contents:

	ticket_type_id	name	price_modifier
1		Single Ride	1.00
2		Full Day Pass	5.00
3		Group Ticket	0.75

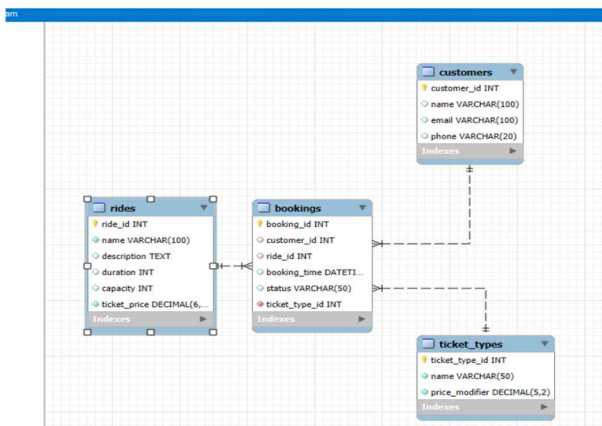
SQL File 13* customers bookings ticket_types rides

1 • SELECT * FROM amusement_park.rides;

Result Grid Filter Rows: Edit Export/Import: Wrap Cell Contents:

	ride_id	name	description	duration	capacity	ticket_price
1		Roller Coaster	High-speed thrilling ride.	5	20	10.00
2		Ferris Wheel	Relaxing panoramic view.	10	40	10.00
3		Haunted House	Spooky dark ride.	7	15	10.00

Result Grid
Form Editor



USER INTERFACE:

Amusement Park Ride Booking System

Available Rides

☐ Roller Coaster (0 min wait)

High-speed thrilling ride.

Duration: 5 mins | Capacity: 20 | Price: \$10.00

☐ Ferris Wheel (0 min wait)

Relaxing panoramic view.

Duration: 10 mins | Capacity: 40 | Price: \$10.00

☐ Haunted House (0 min wait)

Spooky dark ride.

Duration: 7 mins | Capacity: 15 | Price: \$10.00

Customer Details

Name

Email

Phone (optional)

Ticket Type:

Single Ride

Book Ride

Current Bookings

Booking ID	Customer	Email	Phone	Ride	Ticket Type	Booking Time	Status	Cancel
11	Shruthi	shruthi@gmail.com	9176632388	Roller Coaster	Single Ride	8/12/2025, 5:53:34 PM	Booked	Cancel
10	Anuj	anuj@gmail.com	3456789011	Haunted House	Full Day Pass	8/12/2025, 3:58:14 PM	Booked	Cancel
9	Kavya	kavya@gmail.com	1234567890	Ferris Wheel	Group Ticket	8/12/2025, 3:47:42 PM	Booked	Cancel
4	Lucky	lucky@gmail.com	8766543579	Haunted House	Full Day Pass	8/11/2025, 5:25:05 PM	Booked	Cancel
1	sandy	san@gmail.com	897564321	Roller Coaster	Single Ride	8/11/2025, 5:41:49 AM	Booked	Cancel

Ride Analytics

Ride Name	Number of Bookings	Total Revenue (\$)
Roller Coaster	2	20.00
Haunted House	2	20.00
Ferris Wheel	1	10.00

- Available rides disclose each of the rides' specifications
- Customer details allow the customer to book his rides and ticket type by providing his details.
- Current booking updates as the customer's booking is done.
- Ride analytics are the statistical view for number of bookings done and total revenue cost.

SOURCE CODE:

SOURCE CODE: Server.js (Backend)

```
66 // PERFORMING CREATE READ UPDATE DELETE OPERATIONS FOR THE TABLE "Rides"
67 // --- Rides CRUD ---
68
69 // Rides present in the database is shown on the front end.
70 app.get('/rides', (req, res) => {
71   db.query('SELECT * FROM rides', (err, results) => {
72     if (err) {
73       console.error('Error fetching rides:', err);
74       return res.status(500).json({ error: 'Database error fetching rides' });
75     }
76     res.json(results); // if no error, results are printed., //same thing will be viewed with .map in the App.js
77   });
78 });
79 // Updating the booked ride in DB
80 app.post('/rides', (req, res) => {
81   const { name, description, duration, capacity, ticket_price } = req.body;
82   if (!name || !capacity) {
83     return res.status(400).json({ error: 'Name and capacity are required' });
84   }
85   const price = ticket_price !== undefined ? ticket_price : 10.00;
86   const sql = 'INSERT INTO rides (name, description, duration, capacity, ticket_price) VALUES (?, ?, ?, ?, ?)';
87   db.query(sql, [name, description || '', duration || null, capacity, price], (err, results) => {
88     if (err) {
89       console.error('Error creating ride:', err);
90       return res.status(500).json({ error: 'Database error creating ride' });
91     }
92     res.status(201).json({ message: 'Ride created', rideId: results.insertId });
93   });
94 });
95 // Using the ride ids to be combined with other tables
96 app.put('/rides/:id', (req, res) => {
97   const rideId = req.params.id;
98   const { name, description, duration, capacity, ticket_price } = req.body;
99   db.query(
100     'UPDATE rides SET name = ?, description = ?, duration = ?, capacity = ?, ticket_price = ? WHERE ride_id = ?',
101     [name, description, duration, capacity, ticket_price, rideId],
102     (err, results) => {
103       if (err) {
104         console.error('Error updating ride:', err);
105         return res.status(500).json({ error: 'Database error updating ride' });
106       }
107       if (results.affectedRows === 0) {
108         return res.status(404).json({ error: 'Ride not found' });
109       }
110       res.json({ message: 'Ride updated' });
111     }
112   );
113 });
114
115 //
116 app.delete('/rides/:id', (req, res) => {
117   const rideId = req.params.id;
118   db.query('DELETE FROM rides WHERE ride_id = ?', [rideId], (err, results) => {
119     if (err) {
120       console.error('Error deleting ride:', err);
121       return res.status(500).json({ error: 'Database error deleting ride' });
122     }
123     if (results.affectedRows === 0) {
124       return res.status(404).json({ error: 'Ride not found' });
125     }
126     res.json({ message: 'Ride deleted' });
127   });
128 });
129
130 // CRUD OPERATIONS FOR TABLE "Customers"
131 // --- Customers ---
132 // since im not displaying the customer details on the
133 app.post('/customers', (req, res) => {
134   const { name, email, phone } = req.body;
135   if (!name) return res.status(400).json({ error: 'Name is required' });
136   const sql = 'INSERT INTO customers (name, email, phone) VALUES (?, ?, ?)';
137   db.query(sql, [name, email || '', phone || ''], (err, results) => {
138     if (err) {
139       console.error('Error creating customer:', err);
140       return res.status(500).json({ error: 'Database error creating customer' });
141     }
142   });
143 });
```



```

133 app.post('/customers', (req, res) => {
137   db.query(sql, [name, email || '', phone || ''], (err, results) => {
142     res.status(201).json({ message: 'Customer created', customerId: results.insertId });
143   });
144 });
145
146 // --- Ticket Types ---
147
148 app.get('/ticket-types', (req, res) => {
149   db.query('SELECT * FROM ticket_types', (err, results) => {
150     if (err) {
151       console.error('Error fetching ticket types:', err);
152       return res.status(500).json({ error: 'DB error fetching ticket types' });
153     }
154     res.json(results);
155   });
156 });
157
158 // --- Bookings ---
159
160 app.get('/bookings', (req, res) => {
161   const sql = `
162     SELECT b.booking_id, b.booking_time, b.status,
163            c.customer_id, c.name AS customer_name, c.email, c.phone,
164            r.ride_id, r.name AS ride_name, r.description,
165            tt.ticket_type_id, tt.name AS ticket_type_name
166     FROM bookings b
167     JOIN customers c ON b.customer_id = c.customer_id
168     JOIN rides r ON b.ride_id = r.ride_id
169     JOIN ticket_types tt ON b.ticket_type_id = tt.ticket_type_id
170     ORDER BY b.booking_time DESC
171   `;
172   db.query(sql, (err, results) => {
173     if (err) {
174       console.error('Error fetching bookings:', err);
175       return res.status(500).json({ error: 'Database error fetching bookings' });
176     }
177     res.json(results);
178   });

```

SOURCE CODE: App.js (frontend)

```

1  import React, { useEffect, useState } from 'react';
2
3  function App() {
4    const [rides, setRides] = useState([]);
5    const [bookings, setBookings] = useState([]);
6    const [selectedRide, setSelectedRide] = useState(null);
7    const [customerName, setCustomerName] = useState('');
8    const [customerEmail, setCustomerEmail] = useState('');
9    const [customerPhone, setCustomerPhone] = useState('');
10   const [loadingRides, setLoadingRides] = useState(false);
11   const [loadingBookings, setLoadingBookings] = useState(false);
12   const [bookingLoading, setBookingLoading] = useState(false);
13
14   const [waitTimes, setWaitTimes] = useState({});
15   const [analytics, setAnalytics] = useState({});
16   const [loadingAnalytics, setLoadingAnalytics] = useState(false);
17
18   const [ticketTypes, setTicketTypes] = useState([]);
19   const [selectedTicketType, setSelectedTicketType] = useState(1);
20
21   useEffect(() => {
22     fetchRides();
23     fetchBookings();
24     fetchAnalytics();
25     fetchTicketTypes();
26   }, []);
27
28   const fetchRides = () => {
29     setLoadingRides(true);
30     fetch('http://localhost:3001/rides')
31       .then(res => res.json())
32       .then(data => {
33         setRides(data);
34         fetchWaitTimes(data);
35       })
36       .catch(console.error)
37       .finally(() => setLoadingRides(false));
38   };

```

```

40 const fetchWaitTimes = async (ridesList) => {
41   const times = {};
42   for (const ride of ridesList) {
43     try {
44       const res = await fetch('http://localhost:3001/ride-wait-time/${ride.ride_id}');
45       const data = await res.json();
46       times[ride.ride_id] = !data.error ? data.estimated_wait_time_minutes : 0;
47     } catch {
48       times[ride.ride_id] = 0;
49     }
50   }
51   setWaitTimes(times);
52 };
53
54 const fetchBookings = () => {
55   setLoadingBookings(true);
56   fetch('http://localhost:3001/bookings')
57     .then(res => res.json())
58     .then(setBookings)
59     .catch(console.error)
60     .finally(() => setLoadingBookings(false));
61 };
62
63 const fetchAnalytics = () => {
64   setLoadingAnalytics(true);
65   fetch('http://localhost:3001/analytics')
66     .then(res => res.json())
67     .then(data => setAnalytics(data.error ? [] : data))
68     .catch(() => setAnalytics([]))
69     .finally(() => setLoadingAnalytics(false));
70 };
71
72 const fetchTicketTypes = () => {
73   fetch('http://localhost:3001/ticket-types')
74     .then(res => res.json())
75     .then(setTicketTypes)
76     .catch(console.error);
77 };
78
79 const bookRide = async () => {
80   if (!selectedRide) return alert('Please select a ride. ');
81   if (!customerName.trim() || !customerEmail.trim()) return alert('Please enter your name and email. ');
82
83   setBookingLoading(true);
84   try {
85     const responseCustomer = await fetch('http://localhost:3001/customers', {
86       method: 'POST',
87       headers: { 'Content-Type': 'application/json' },
88       body: JSON.stringify({ name: customerName.trim(), email: customerEmail.trim(), phone: customerPhone.trim() }),
89     });
90     if (!responseCustomer.ok) throw new Error((await responseCustomer.json()).error || 'Failed to create customer');
91     const customerData = await responseCustomer.json();
92
93     const responseBooking = await fetch('http://localhost:3001/bookings', {
94       method: 'POST',
95       headers: { 'Content-Type': 'application/json' },
96       body: JSON.stringify({
97         customer_id: customerData.customerId,
98         ride_id: selectedRide,
99         booking_time: new Date().toISOString(),
100         status: 'Booked',
101         ticket_type_id: selectedTicketType,
102       }),
103     });
104
105     if (!responseBooking.ok) throw new Error((await responseBooking.json()).error || 'Failed to create booking');
106     const bookingData = await responseBooking.json();
107
108     alert(`Booking successful! Booking ID: ${bookingData.bookingId}`);
109
110     setCustomerName('');
111     setCustomerEmail('');
112     setCustomerPhone('');
113     setSelectedRide(null);
114     setSelectedTicketType(1);

```

```

115     fetchBookings();
116     fetchRides();
117   } catch (error) {
118     alert('Error booking ride: ${error.message}');
119   } finally {
120     setBookingLoading(false);
121   }
122 };
123
124
125 const cancelBooking = async (bookingId) => {
126   if (!window.confirm('Are you sure you want to cancel this booking?')) return;
127   try {
128     const res = await fetch('http://localhost:3001/bookings/${bookingId}', { method: 'DELETE' });
129     if (!res.ok) throw new Error((await res.json()).error || 'Failed to cancel booking');
130     alert('Booking canceled.');
```

fetchBookings();

fetchRides();

} catch {

alert('Error canceling booking');

}

};

const appWrapperStyle = {

minHeight: '100vh',

background: 'linear-gradient(135deg, #6a11cb 0%, #2575fc 100%)',

padding: '40px 20px',

boxSizing: 'border-box',

};

const containerStyle = {

maxWidth: 1000,

margin: '0 auto',

fontFamily: "'Segoe UI', Tahoma, Geneva, Verdana, sans-serif",

padding: '30px 40px',

backgroundColor: 'rgba(255, 255, 255, 0.95)',

```

349     <h2 style={{ borderBottom: '2px solid #28a745', paddingBottom: 8 }}>
350       Ride Analytics {loadingAnalytics && '(Loading...)}
351     </h2>
352     {loadingAnalytics ? (
353       <p>Loading analytics data...</p>
354     ) : analytics.length > 0 ? (
355       <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: 14 }}>
356         <thead>
357           <tr style={{ backgroundColor: '#28a745', color: 'white' }}>
358             <th style={{ padding: 10, border: '1px solid #ddd' }}>Ride Name</th>
359             <th style={{ padding: 10, border: '1px solid #ddd' }}>Number of Bookings</th>
360             <th style={{ padding: 10, border: '1px solid #ddd' }}>Total Revenue ($)</th>
361           </tr>
362         </thead>
363         <tbody>
364           {analytics.map((item, index) => (
365             <tr key={index} style={{ backgroundColor: index % 2 === 0 ? 'white' : '#e6ffe6' }}>
366               <td style={{ padding: 10, border: '1px solid #ddd' }}>{item.ride_name}</td>
367               <td style={{ padding: 10, border: '1px solid #ddd' }}>{item.booking_count}</td>
368               <td style={{ padding: 10, border: '1px solid #ddd' }}>{item.total_revenue}</td>
369             </tr>
370           ))}
371         </tbody>
372       </table>
373     ) : (
374       <p>No analytics data available yet.</p>
375     )}
376   </section>
377 </div>
378 </div>
379 );
380 }
381
382 export default App;
383
384
385
```